RANK, DENSE_RANK, CTE & PARTITION BY

```
-- 1. CREATE SEPARATE DATABASE
CREATE DATABASE sql_practice;
-- MySQL
USE sql_practice;
-- 2. CREATE EMPLOYEES TABLE
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   employee_name VARCHAR(50),
   department VARCHAR(50),
   salary INT
);
-- 3. INSERT SAMPLE DATA
INSERT INTO employees
(employee_id, employee_name, department, salary)
VALUES
(1, 'Arun',    'IT',      50000),
(2, 'Priya',   'IT',      60000),
(3, 'Rahul',   'IT',      60000),
(4, 'Sneha',   'IT',      40000),
(5, 'Kiran',   'HR',      55000),
(6, 'Divya',   'HR',      55000),
(7, 'Ravi',    'HR',      45000),
(8, 'Vijay',   'Finance', 70000),
(9, 'Meena',   'Finance', 65000),
(10,'Karthik','Finance', 65000),
(11,'Anu',     'Finance', 50000),
(12,'Suresh', 'DevOps',  80000),
(13,'Deepa',   'DevOps',  75000),
(14,'Manoj',   'DevOps',  75000),
(15,'Nisha',   'DevOps',  60000);
-- 4. CHECK THE DATA
SELECT * FROM employees;
```

1. RANK = “SAME VALUE, SAME RANK, THEN SKIP”

Think:

If salary is the same → give same rank.
After duplicate rank → skip the next rank.

```
SELECT
   name,
   salary,
     RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Result:

```
name    salary   salary_rank
-----------------------------
Priya   60000       1
Rahul   60000       1
Kiran   55000       3
Divya   55000       3
Arun    50000       5
```

Ravi 45000 6
Sneha 40000 7

Why?

60000 → Rank 1
60000 → Rank 1
55000 → Rank 3 <-- Rank 2 skipped
55000 → Rank 3
50000 → Rank 5 <-- Rank 4 skipped

Remember:

RANK = SAME RANK + SKIP

2. DENSE_RANK = "SAME VALUE, SAME RANK, NO SKIP"

Think:

If salary is the same → give same rank.
But don't skip any rank.

```
SELECT
   name,
   salary,
     DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Result:

```
name    salary   salary_rank
-----------------------------
```

Priya 60000 1
Rahul 60000 1
Kiran 55000 2
Divya 55000 2
Arun 50000 3
Ravi 45000 4
Sneha 40000 5

Remember:

```
DENSE_RANK = SAME RANK + NO SKIP
```

The easiest difference is:

```
Salary    RANK    DENSE_RANK
-------------------------------
```

60000 1 1
60000 1 1
55000 3 2
55000 3 2
50000 5 3

So in an interview, you can simply say:

"RANK and DENSE_RANK both give the same rank for duplicate values. The main difference is RANK skips the next rank, but DENSE_RANK doesn't skip the rank."

Hands-on: Find 2nd highest salary

This is a very common interview question.

Think:

```
DENSE_RANK the salary → then take rank 2.
WITH ranked_employees AS
(
```

```
    SELECT
    name,
    salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT *
FROM ranked_employees
WHERE rnk = 2;
```

Result:

Kiran 55000 2

Divya 55000 2

Why two employees?

Because both have the second-highest salary of 55000.

3. CTE = "CREATE TEMPORARY RESULT, THEN USE IT"

CTE means:

Common Table Expression

Don't worry about the name.

Think simply:

First create a temporary result → then write SQL on top of that result.

Basic syntax:

```
WITH temp_table AS
(
    SELECT ...
)
SELECT *
FROM temp_table;
```

Remember:

CTE = TEMP RESULT FOR MY QUERY

For example, I want employees earning more than 50,000.

```
WITH high_salary AS
(
    SELECT
    name,
    salary
    FROM employees
    WHERE salary > 50000
)
SELECT *
FROM high_salary;
```

First this part runs:

```
SELECT
  name,
  salary
FROM employees
WHERE salary > 50000;
```

It produces:

Priya 60000

Rahul 60000

Kiran 55000

Divya 55000

We are giving this result a name:

`high_salary`

Then:

```
SELECT *
FROM high_salary;
```

So remember:

CTE = CREATE RESULT → GIVE NAME → USE RESULT

A very important real interview example is 2nd highest salary department-wise.

Here we use all concepts together:

```
WITH ranked_employees AS
(
    SELECT
    name,
    department,
    salary,
        DENSE_RANK() OVER
        (
            PARTITION BY department
            ORDER BY salary DESC
    ) AS rnk
    FROM employees
)
SELECT *
FROM ranked_employees
WHERE rnk = 2;
```

Here:

`PARTITION BY department`

means:

Do ranking separately for each department.

So remember the full concept like this:

`RANK`

Same value → Same rank → SKIP

`DENSE_RANK`

Same value → Same rank → NO SKIP

`CTE`

Create temporary result → Give it a name → Use it

`PARTITION BY`

Do the calculation separately for each group

For interview practice, the most important hands-on questions after this are ROW_NUMBER vs RANK vs DENSE_RANK, 2nd/3rd highest salary, top 3 salaries department-wise, duplicate removal using ROW_NUMBER, and CTE vs subquery.